

Fundamentos de programación

Alex Di Genova

08/09/2026

Why learn to code in Python?

- Python code is designed to be readable, and hence reusable and maintainable.
 - Easy to learn, understand and remember.
- Developer productivity (1/3 of C or C++ code).
- Program portability (run on major operating systems)
- A large collection of libraries.
- Component integration (C and C++)
- A big community.
- Python is a general-purpose programming language that blends procedural, functional, and object-oriented paradigms.
- Speed is a downside of Python (10-100X slower than C/C++).



- What can I do?
 - System programming.
 - GUIs
 - Internet scripting
 - Database programming
 - Rapid prototyping
 - Numeric and scientific programming
 - Machine learning.

First code in Python

Python

```
1 #file.py
2 print("Hello World")
```

Code



Interpretation



First code in Python

Python

```
1 #file.py~
2 print("Hello World")~
```

Bash

```
1 #!/bin/bash~
2 echo "Hello, World"~
```

Java

```
1 //file.java~
2 class HelloWorld~
3 {~
4     public static void main(String args[])~
5     {~
6         System.out.println("Hello, World");~
7     }~
8 }
```

C

```
1 #include<stdio.h>~
2 int main()~
3 {~
4     printf("Hello, World");~
5     return 0;~
6 }
```

C++

```
1 //file.cpp~
2 #include<iostream>~
3 int main()~
4 {~
5     std::cout<<"Hello, World!"<<std::endl;~
6     return 0;~
7 }
```

R

```
1 hello<- "Hello, World!"~
2 print(hello)~
3 ~
```

Code



Interpretation



First code in Python



¿Qué frases entienden los computadores?

Operaciones matemáticas

- En matemáticas, un **operador** es un símbolo que se usa cuando resolvemos un problema matemático.

- + (sumar)
- - (restar)
- x (multiplicar)
- ÷ (division)
- // (division entera)
- % (modulo)
- Exp (exponencial)

```
[ 3 ] 1 # suma 13  
      2 6 + 7 + 4 + 5
```

22

```
[ ] 1 # resta  
    2 10 - 5
```

5

```
[ ] 1 # multiplicacion 18  
    2 3 * 6
```

18

```
▶ 1 # division  
  2 8 / 3
```

▶ 2.6666666666666665

```
[ ] 1 # division entera  
    2 12//3
```

4

```
▶ 1 # modulo  
  2 10 % 5
```

▶ 0

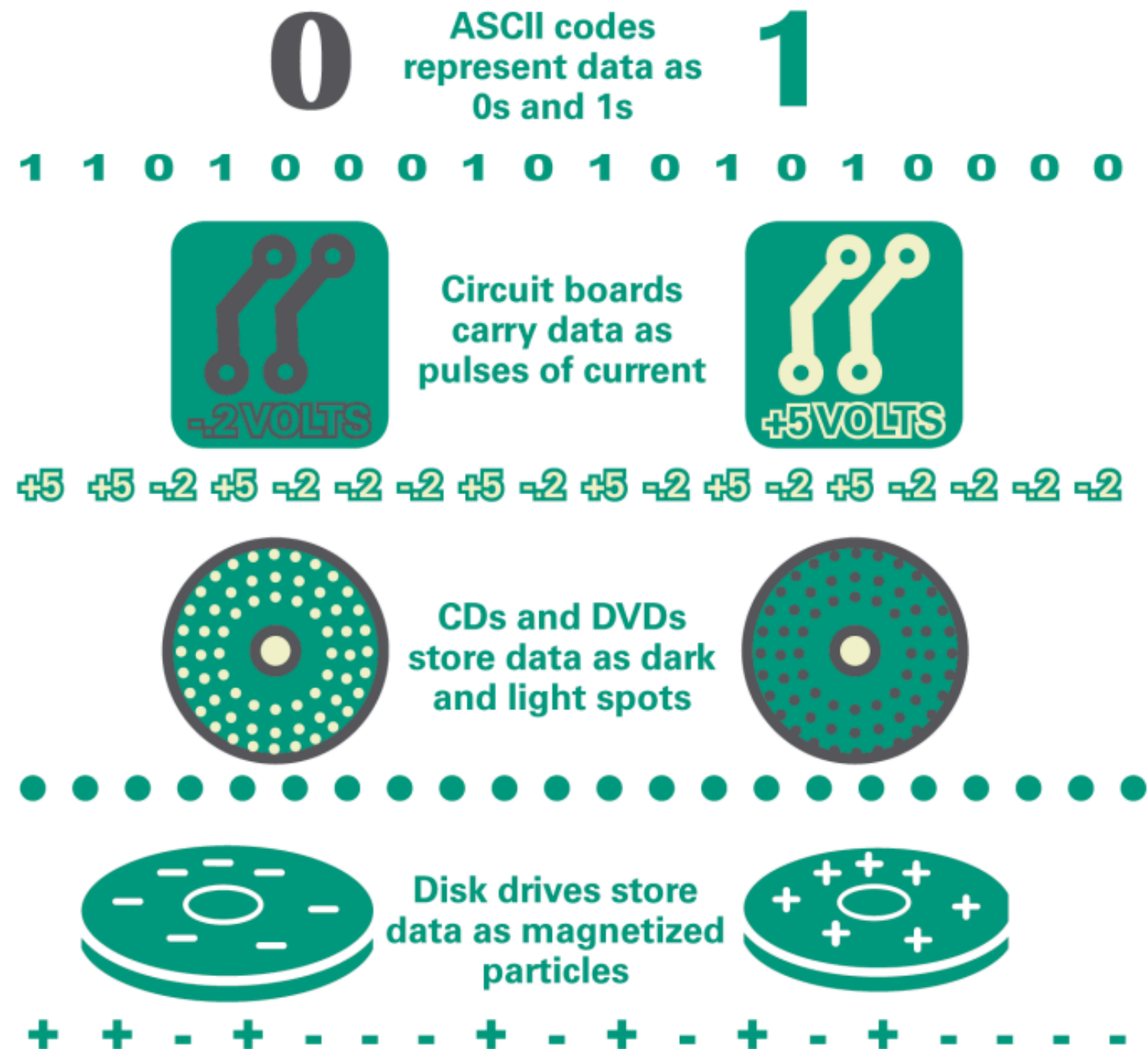
```
[ 4 ] 1 # exponencial 2 * 2 * 2  
      2 5 ** 3
```

125

¿Qué frases entienden los computadores?

¿Cómo representa la información mi computador?

- Los **0** y **1** utilizados para representar datos se denominan dígitos binarios. Este término obtuvo el nombre **bit** que significa **dígito binario**.
- Un **bit** es un **0** o un **1**, la representación digital de la información.
- Un archivo digital (archivo) es una colección de datos que existe en un medio de almacenamiento, como el **disco duro**, **CD**, **DVD** o **memoria flash**.



¿Qué frases entienden los computadores?

Representando números

- Los **computadores** representan datos numéricos usando el sistema numérico binario, también llamado **base 2**
- El sistema numérico binario solo tiene dos dígitos: **0 y 1**.
- En python: usamos las funciones **bin** y **int** para convertir números a binario y de binarios a números.

ASCII extendido utiliza ocho bits para cada carácter (2^8 caracteres).

A es 01000001

```
0 -> 0b0 -> 0
1 -> 0b1 -> 1
2 -> 0b10 -> 2
3 -> 0b11 -> 3
4 -> 0b100 -> 4
5 -> 0b101 -> 5
6 -> 0b110 -> 6
7 -> 0b111 -> 7
8 -> 0b1000 -> 8
9 -> 0b1001 -> 9
10 -> 0b1010 -> 10
11 -> 0b1011 -> 11
12 -> 0b1100 -> 12
13 -> 0b1101 -> 13
14 -> 0b1110 -> 14
15 -> 0b1111 -> 15
16 -> 0b10000 -> 16
17 -> 0b10001 -> 17
18 -> 0b10010 -> 18
19 -> 0b10011 -> 19
20 -> 0b10100 -> 20
21 -> 0b10101 -> 21
22 -> 0b10110 -> 22
23 -> 0b10111 -> 23
24 -> 0b11000 -> 24
25 -> 0b11001 -> 25
26 -> 0b11010 -> 26
27 -> 0b11011 -> 27
28 -> 0b11100 -> 28
29 -> 0b11101 -> 29
30 -> 0b11110 -> 30
31 -> 0b11111 -> 31
```

```
a = bin(5)
print(a)
print(int(a))
```

1 1 0 1 1 0 1 1

Binary to Decimal

1 x 2⁰ = 1 x 1 = 1
1 x 2¹ = 1 x 2 = 2
0 x 2² = 0 x 4 = 0
1 x 2³ = 1 x 8 = 8
1 x 2⁴ = 1 x 16 = 16
0 x 2⁵ = 0 x 32 = 0
1 x 2⁶ = 1 x 64 = 64
1 x 2⁷ = 1 x 128 = 128

1 + 2 + 8 + 16 + 64 + 128 = 219

(11011011) = (219)

¿Qué frases entienden los computadores?

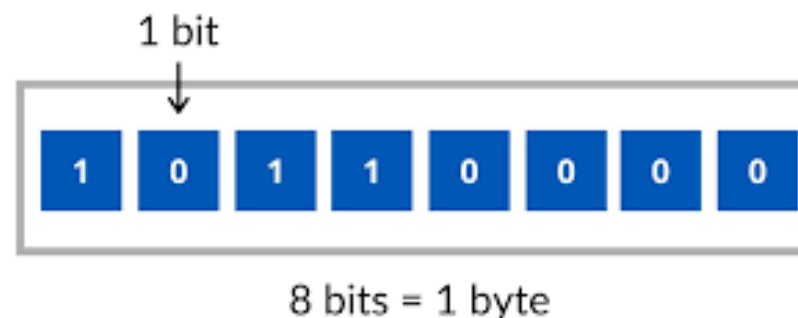
Representando texto ASCII

Alfabeto com os símbolos de puntuación e espacios	00100000	Space	00110011	3	01000110	F	01011001	Y	01101100	l
	00100001	!	00110100	4	01000111	G	01011010	Z	01101101	m
	00100010	"	00110101	5	01001000	H	01011011	[	01101110	n
	00100011	#	00110110	6	01001001	I	01011100	\	01101111	o
	00100100	\$	00110111	7	01001010	J	01011101	]	01110000	p
	00100101	%	00111000	8	01001011	K	01011110	^	Python:	q
	00100110	&	00111001	9	01001100	L	01011111	_	01110010	r
	00100111	'	00111010	:	01001101	M	01100000	`	#texto a binario	
	00101000	(	00111011	;	01001110	N	01100001	a	print("A")	
	00101001	)	00111100	<	01001111	O	01100010	b	print(ord("A"))	
	00101010	*	00111101	=	01010000	P	01100011	c	print(bin(ord("A")))	
	00101011	+	00111110	>	01010001	Q	01100100	d	print(chr(ord("A")))	
	00101100	,	00111111	?	01010010	R	01100101	e	01110110	v
	00101101	-	01000000	@	01010011	S	01100110	f	01110111	w
	00101110	.	01000001	A	01010100	T	01100111	g	65	x
	00101111	/	01000010	B	01010101	U	01101000	h	0b1000001	y
	00110000	0	01000011	C	01010110	V	01101001	i	01110001	z
	00110001	1	01000100	D	01010111	W	01101010	j	01111010	{
	00110010	2	01000101	E	01011000	X	01101011	k	01111011	
	00110011	3	01000110	F	01011001	Y	01101100	l	01111100	}

¿Qué frases entienden los computadores?

Bits y bites

Todos los datos almacenados y transmitidos por computadores se codifican como **bits**.



UNIDAD	SÍMBOLO	EQUIVALENCIA		
Kilobyte	kB	2 ¹⁰ bytes	1024 bytes	
Megabyte	MB	2 ¹⁰ kilobytes	1024 kilobytes	
Gigabyte	GB	2 ¹⁰ megabytes	1024 megabytes	1000.000.000 bytes
Terabyte	TB	2 ¹⁰ gigabytes	1024 gigabytes	1 billón de bytes
<u>Petabyte</u>	PB	2 ¹⁰ terabytes	1024 terabytes	1000 billones de bytes
Exabyte	EB	2 ¹⁰ <u>petabytes</u>	1024 <u>petabytes</u>	1 trillón de bytes
<u>Zettabyte</u>	ZB	2 ¹⁰ exabytes	1024 exabytes	1000 trillones de bytes
<u>Yottabyte</u>	YB	2 ¹⁰ <u>zettabytes</u>	1024 <u>zettabyte</u>	1 cuatrillón de bytes

¿Qué frases entienden los computadores?

Explicación del concepto variable.

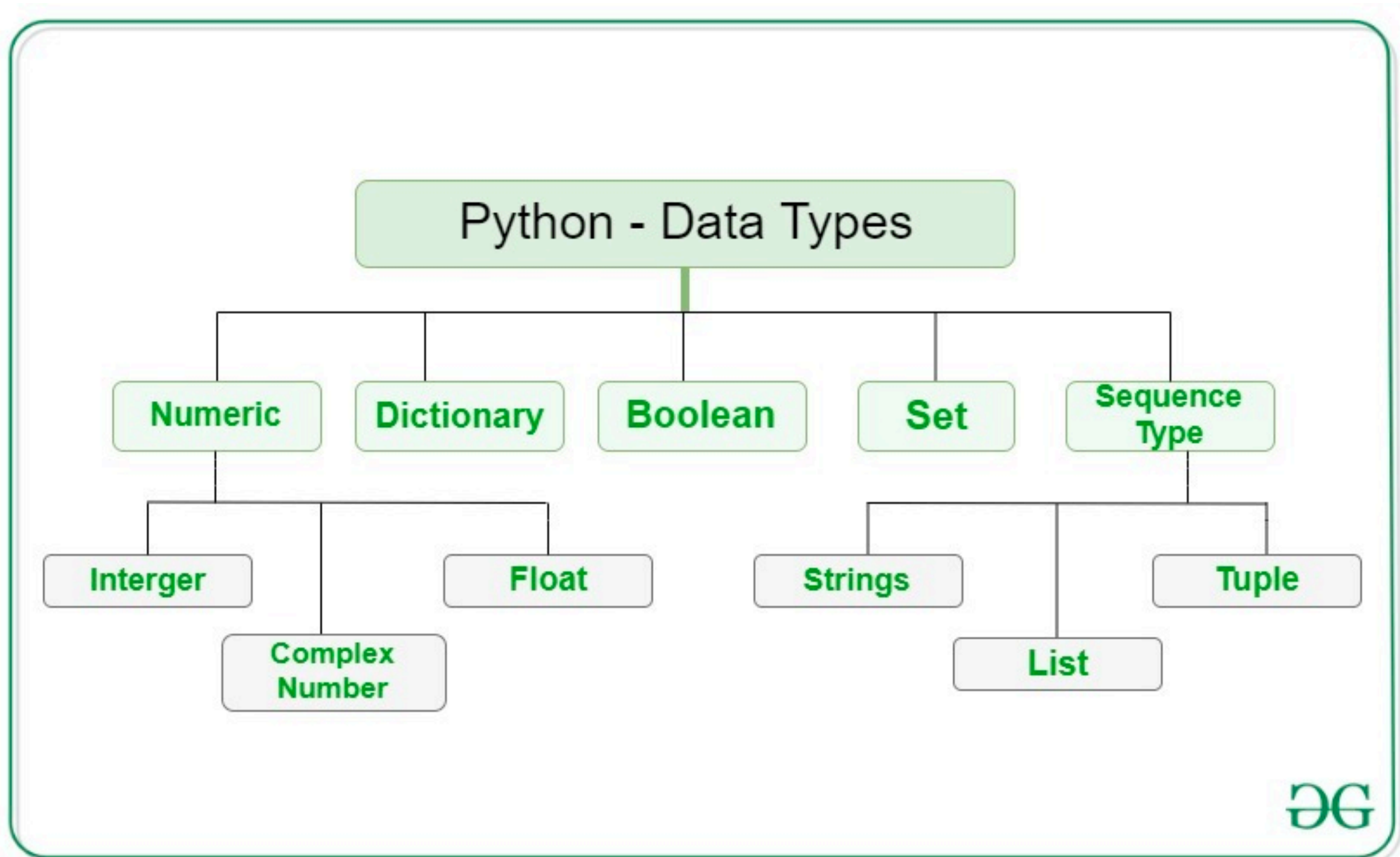
Las variables tiene dos partes:

1. **El nombre.** También llamado identificador: algo que se usa para identificar (decir qué es algo). El nombre de la variable se utiliza para identificar un dato exacto.
 2. **El valor.** Valor significa cantidad o tipo. La parte de valor de una variable muestra los datos que necesitamos para realizar un seguimiento.
- Usamos un signo igual (=) para asignar (dar) un **valor** al **nombre** de una **variable**.

```
# variables
a = 5
print(a)
a = 7
print(a)
b = a
print(b)
c = "texto"
b = c
print(b)
```

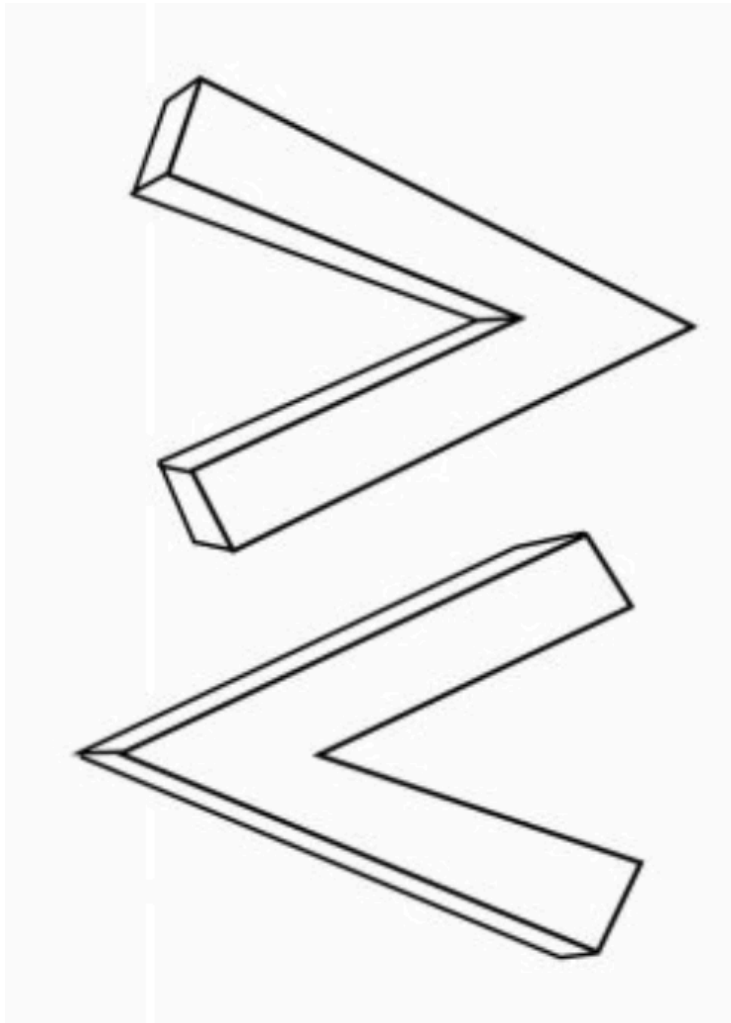
¿Qué frases entienden los computadores?

Explicación del concepto variable.



Mi computador es el mejor para el juego de la verdad

Comparaciones numéricas (mayor, menor, igual) en Python.



```
1 5 > 7
```

False

```
1 5 < 7
```

True

```
1 a = 100
2 b = 50
3 a < b
```

False

```
1 5 == 5
```

True

```
[66] 1 5 == 6
```

False

```
[67] 1 5 is 5
```

True

```
1 5 is 6
```

False

```
1 5 > 3 and 10 < 20
```

True

```
1 a = 5
2 a is 5 or a == 4
```

True

```
1 a >= 5
```

True

```
1 a <= 10
```

True

```
1 a = "Alex"
2 b = "Carol"
3 a != b
```

True

```
1 a is not b
```

True

Mi computador es el mejor para el juego de la verdad

Manejo y comparaciones de palabras en Python.

- Las palabras (strings) se utilizan para almacenar texto.

```
[1] 1 S = 'palabra'
     2 len(S)
```

7

- En python se implementa una colección/arreglo o por posición de objetos (orden de izquierda-derecha).

```
[7] 1 print(S[0]) #primer caracter de la secuencia
     2 print(S[3]) # cuarto caracter
```

p
a

- Strings son secuencias de carácter (listas, tuplas).

```
▶ 1 # indices de secuencias comienzan en 0 .. n-1
   2 print(S[-1]) # ultimo caracter
   3 print(S[-2]) # ante-penultimo caracter
```

Strings contienen índices por posición.

a
r

Mi computador es el mejor para el juego de la verdad

Manejo y comparaciones de palabras en Python.

- Podemos extraer porciones de un string [pos1:pos2).

```
print(S[2:5])    lab
print(S[1:])     alabra
print(S[:4])     pala
print(S[:-1])    palabr
print(S[:])      palabra
```

- Concatenar strings

```
S + ' dos '
```

- Repetir strings

```
S * 4
```

- Los string son inmutables.

```
S[0] = "P"      S = 'P' + S[1:]
print(S)        Palabra
```

Mi computador es el mejor para el juego de la verdad

Manejo y comparaciones de palabras en Python.

- Los string poseen funciones específicas.

capitalize casefold center count encode **endswith** expandtabs **find**
format format_map index isalnum isalpha **isascii** **isdecimal**
isdigit
isidentifier islower isnumeric isprintable isspace istitle
isupper **join**
ljust lower lstrip maketrans partition replace rfind rindex
rjust rpartition rsplit rstrip **split** splitlines **startswith** strip
swapcase title translate **upper** zfill

```
print(S.capitalize())  
print(S.upper())  
print(S.find("bra"))  
print(S.replace("bra", "BRA"))  
l="aaa,bbb,ccc,ddd\n"  
print(l.split(","))  
print(l.rstrip())  
print(l.rstrip().split(","))
```

```
Palabra  
PALABRA  
4  
PalaBRA  
['aaa', 'bbb', 'ccc', 'ddd\n']  
aaa,bbb,ccc,ddd  
['aaa', 'bbb', 'ccc', 'ddd']
```

A mi computador le gusta tomar decisiones

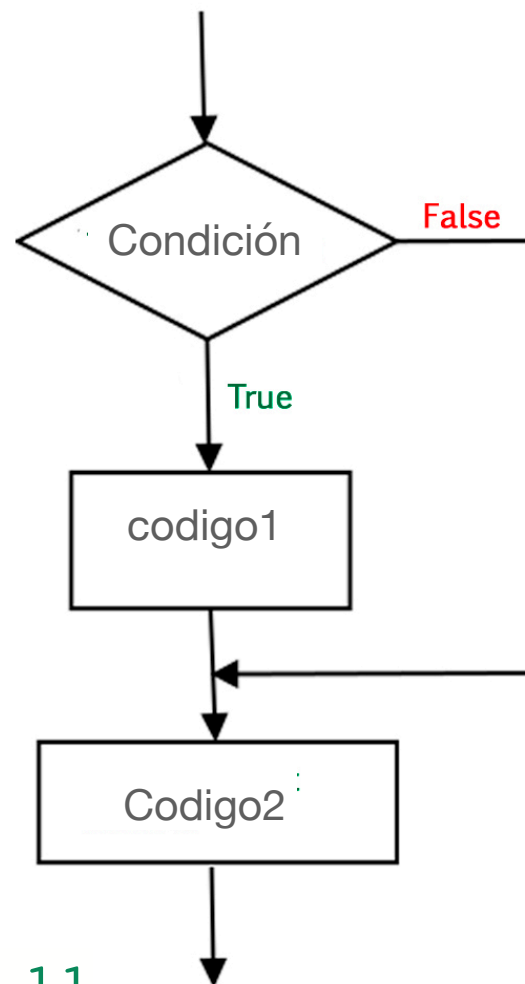
Presentación de instrucciones de decisión (if, elif, else) en Python



A mi computador le gusta tomar decisiones

Presentación de instrucciones de decisión (if, elif, else) en Python

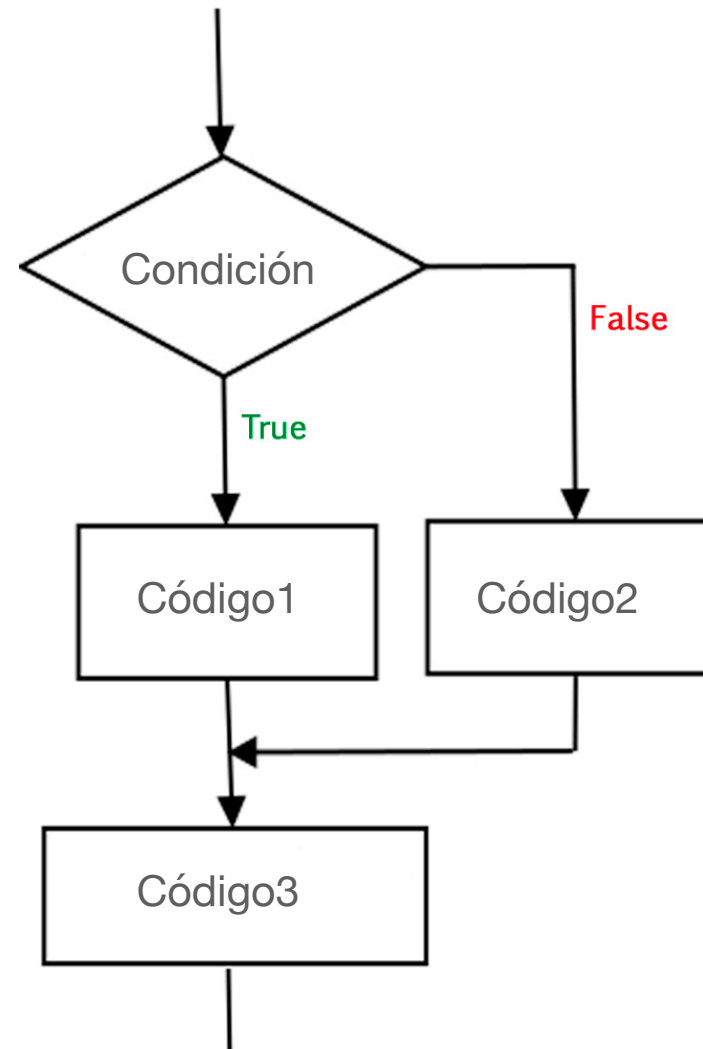
IF



```
a = 11
if(a > 10):
    print("el valor es mayor a 10")

print("el valor es:", a)
```

IF—ELSE

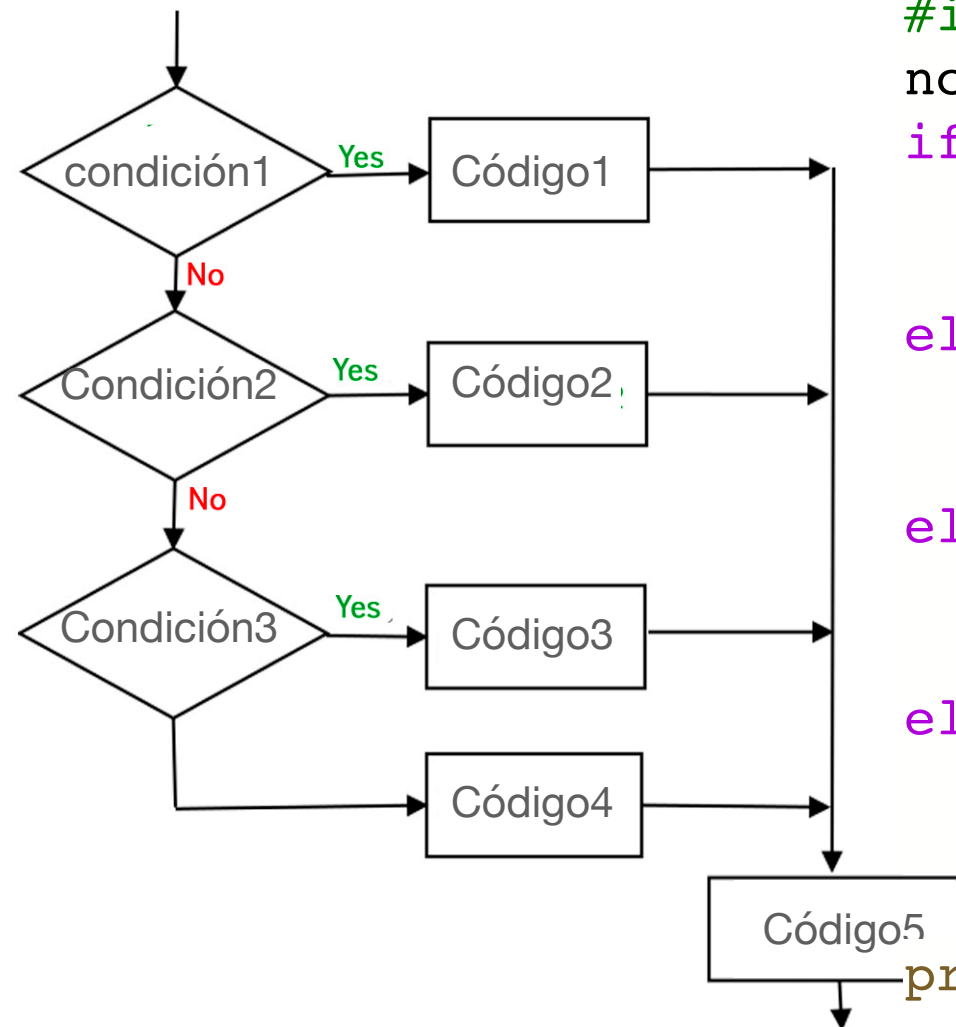


```
texto = "Alex"
if(texto == "Alex"):
    print("Hola Alex")
else:
    print("No eres alex")
```

A mi computador le gusta tomar decisiones

Presentación de instrucciones de decisión (if, elif, else) en Python

IF-elif-else



```
#if-elif-else
nombre= "Alex"
if(nombre == "Alex"):
    print("Hola Alex")
    print("Se que te gusta jugar al futbol")
elif(nombre == "Roberto"):
    print("Hola Roberto")
    print("Me contaron que te gusta leer libros")
elif(nombre == "Camila"):
    print("Hola Camila")
    print("Se que te gusta programar")
else:
    print("Hola ",nombre)
    print("No se nada de ti")
print("Bienvenido")
```

A mi computador le gusta tomar decisiones

Presentación de comparaciones con múltiples decisiones.

```
edad = 20
nombre = "Carlos"
ciudad = "Talca"
```

```
if(nombre == "Carlos"):
    if(ciudad == "Rancagua"):
        if(edad < 20):
            print("Hola Carlos de Rancagua, tienes menos de 20 años")
        else:
            print("Hola Carlos de Rancagua, tienes más de 20 años")
    else:
        print("No es Carlos, pero es de Rancagua")
else:
    edad = 20
    nombre = "Alex"
    ciudad = "Rancagua"
```

Que

```
if(nombre == "Carlos" and ciudad == "Rancagua"):
    if(edad < 20):
        print("Hola Carlos de Rancagua, tienes menos de 20 años")
    else:
        print("Hola Carlos de Rancagua, tienes más de 20 años")
else:
    if(nombre == "Carlos"): print("es Carlos pero no es de Rancagua")
    if(ciudad == "Rancagua"): print("No es Carlos, pero es de Rancagua")
```

A mi computador le gusta tomar decisiones

Controlando el tiempo en el computador.

`sleep(...)`
`sleep(segundos)`

Retrasar la ejecución durante un número determinado de segundos. El argumento puede ser un número de punto flotante para una precisión de subsegundos.

```
import time

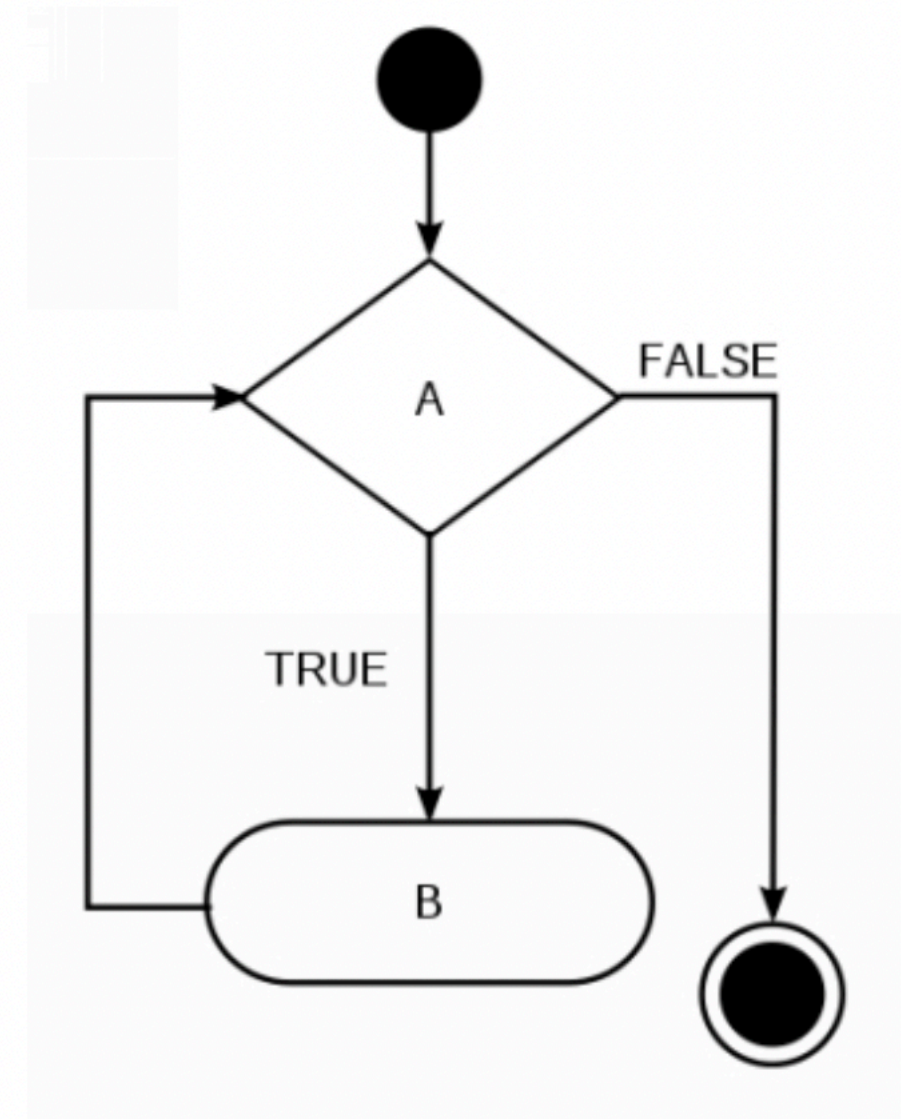
a = 10
nombre = input("Ingresa un nombre:")
if(nombre == "Roberto"):
    time.sleep(10)
    print("Roberto esta esperando")
else:
    time.sleep(1)
    print("Hola", nombre)
```

Que hace este código?

Mi computador es el mejor para repetir instrucciones.

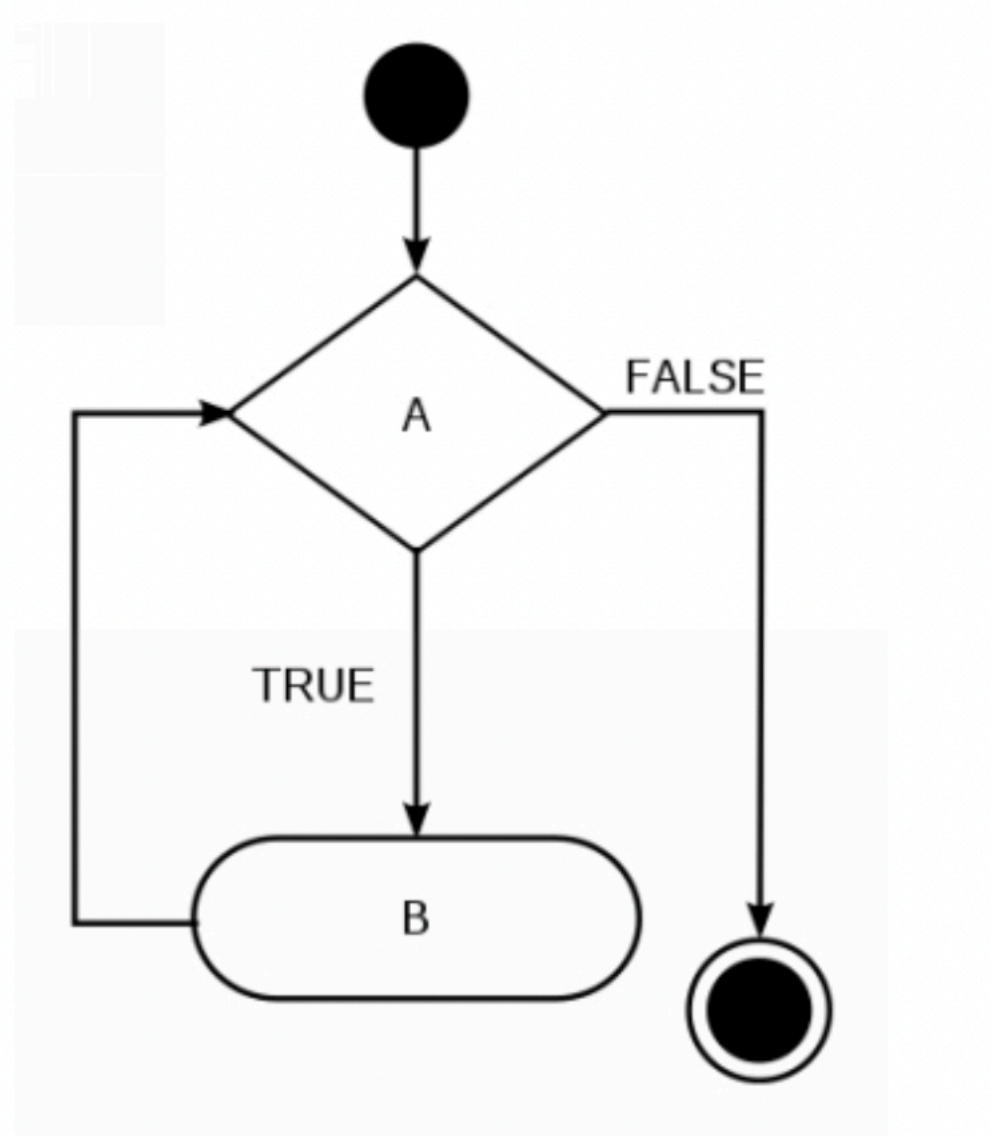
Presentación de instrucción de iteración “While” en Python

- Una computadora puede seguir haciendo lo mismo una y otra vez para siempre.
- Un bucle (loop) es un comando para repetir algo hasta que suceda algo más. Da vueltas y vueltas, una y otra vez, como un círculo.
- Uno de los tipos de bucles más populares es el bucle **while**.
 - Mientras _____ es Verdadero, haz _____



Mi computador es el mejor para repetir instrucciones.

Presentación de instrucción de iteración “While” en Python



```
while 2 > 1:  
    print("Hola --- no puedo parar...")
```

```
i = 1  
while i < 5:  
    print(i)
```

```
i = 1  
while i < 5:  
    print(i)  
    i=i+1
```

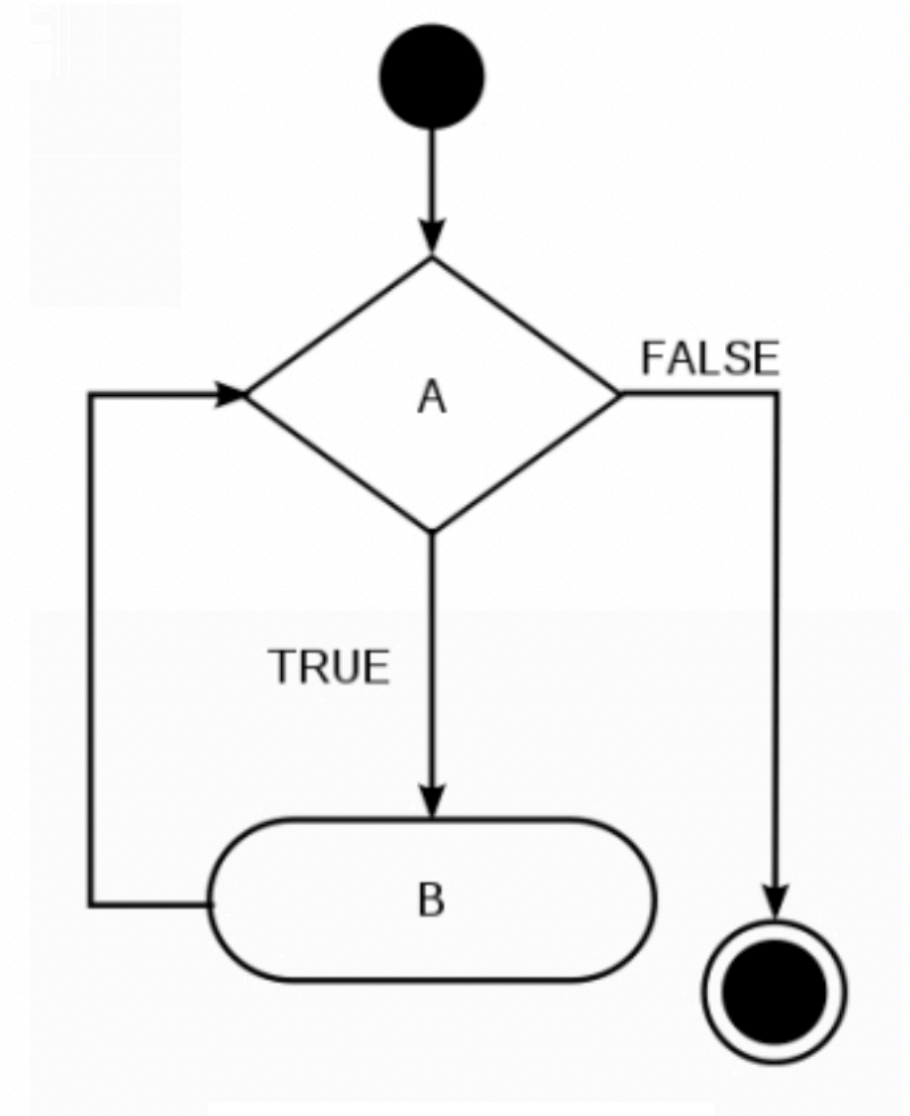
```
i = 1  
while i > 5:  
    print(i)  
    i=i+1
```

```
i = 1  
while i < 5:  
    print(i)  
    i=i+1  
i=0
```

Mi computador es el mejor para repetir instrucciones.

Presentación de instrucción de iteración “For” en Python

- Con el ciclo **while**, no especificamos (decimos claramente) cuántas veces repetir el ciclo. No dijimos, repite 5 veces..
- Con el ciclo **for**, especificamos cuántas veces repetir el ciclo.
- Una de las funciones que podemos usar con bucles for es **range()**. La función **range()** se puede usar para crear una lista de números dentro de un cierto rango.



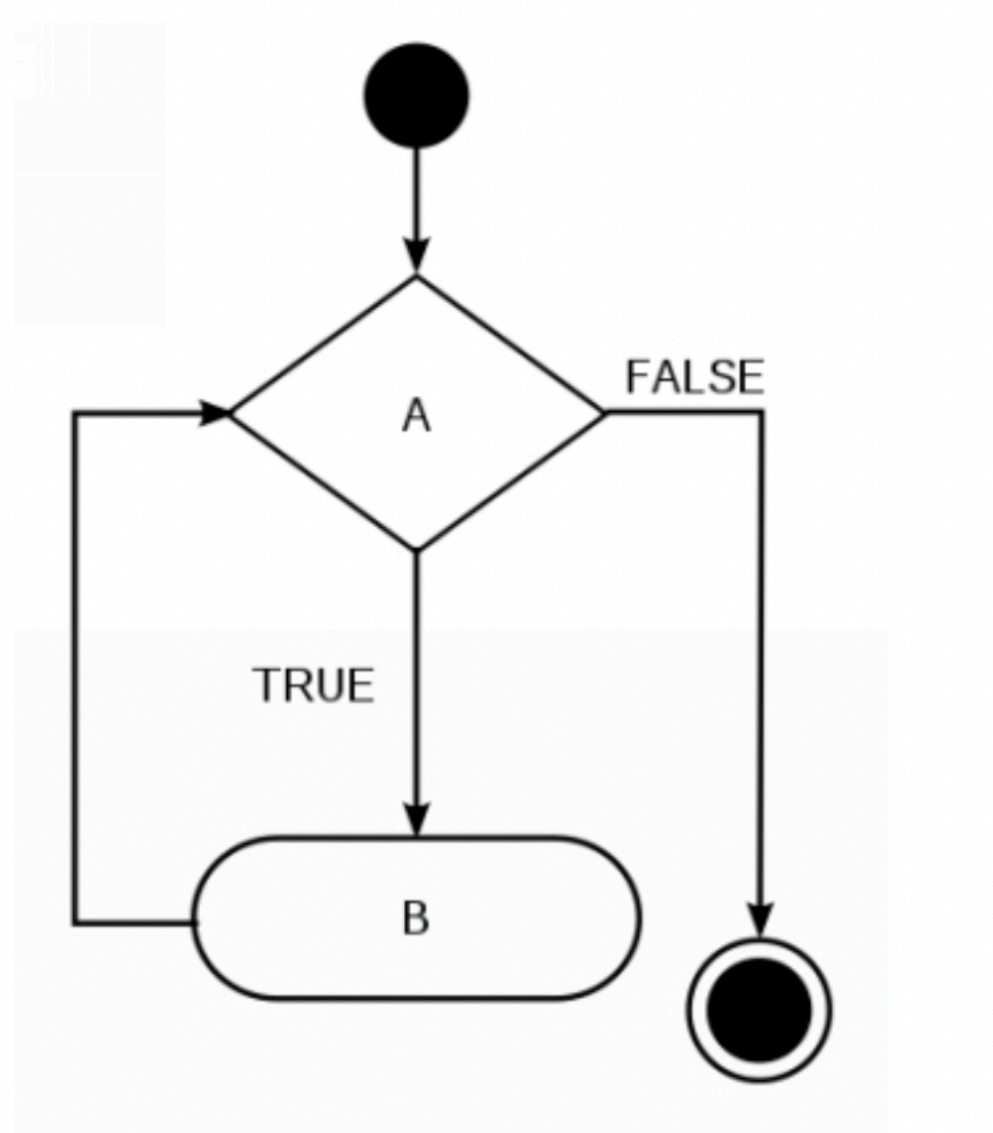
range(3)



[0,1,2]

Mi computador es el mejor para repetir instrucciones.

Presentación de instrucción de iteración “For” en Python



```
for i in range(1,10):  
    print(i)
```

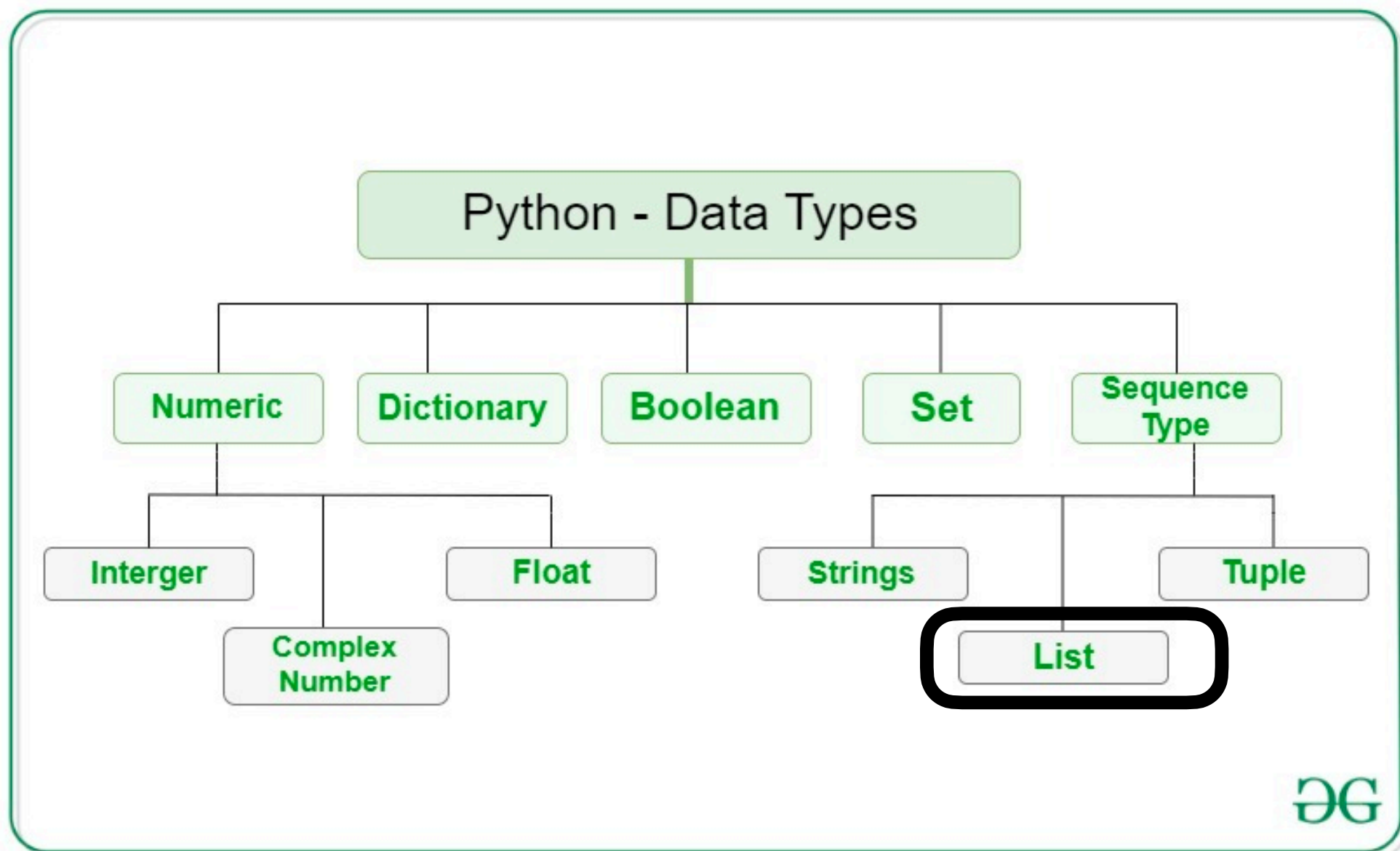
```
S="palabra"  
for i in S:  
    print(i)
```

```
for i in S:  
    for j in range(10):  
        print(i,j)
```

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Tipos de variables en python:



Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

- Una lista, se define como una colección de elementos en un orden determinado.
- Ejemplos: letras del alfabeto, dígitos del 0 al 9, o los nombres de todas las personas de tu familia.
- Sin embargo, los elementos de una lista no tienen que estar necesariamente relacionados de ninguna manera en particular.



Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Creando una lista



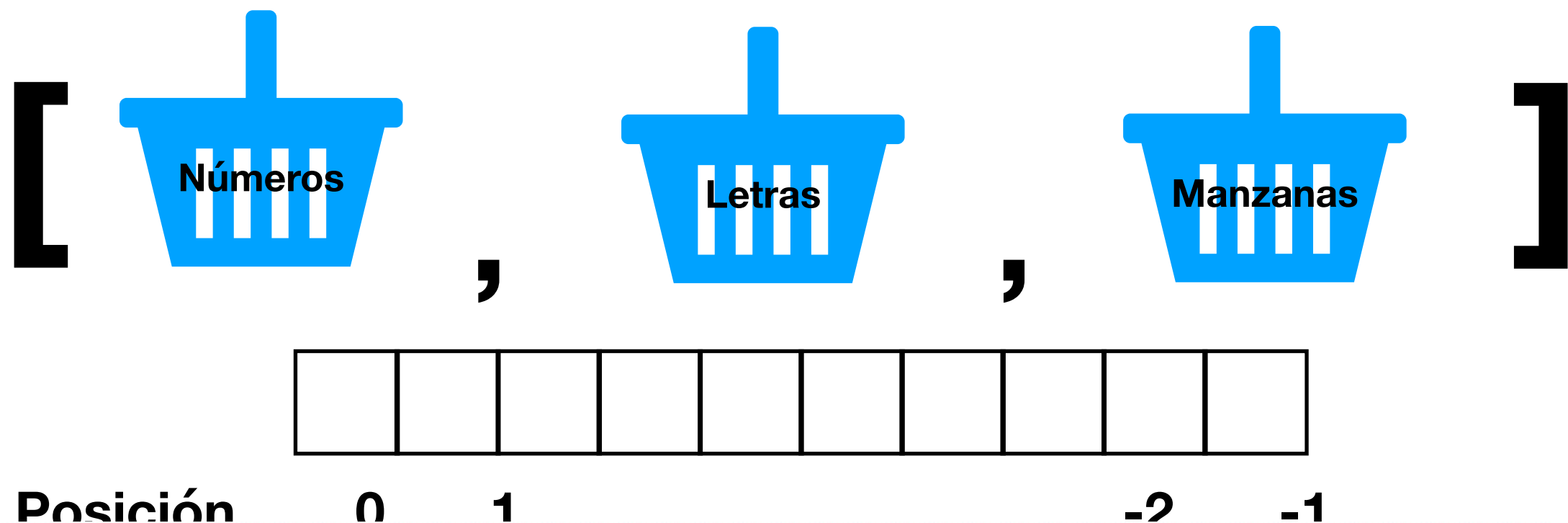
```
frutas = ['manzanas', 'peras', 'platanos', 'frutillas']  
print(frutas)
```

```
['manzanas', 'peras', 'platanos', 'frutillas']
```

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Como accedemos a los elementos en una lista?



```
# acceso a los elementos en una lista
frutas = ['manzanas', 'peras', 'platanos', 'frutillas']
mensaje = "Mi fruta favorita son las/los " + frutas[0].title() + "."
print(mensaje)
```

➤ Mi fruta favorita son las/los Manzanas.

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Como modificamos los elementos en una lista?



```
# modificar elementos en una lista
frutas = ['manzanas', 'peras', 'platanos', 'frutillas']
print(frutas)
frutas[3] = "naranjas"
print(frutas)
```

```
['manzanas', 'peras', 'platanos', 'frutillas']
['manzanas', 'peras', 'platanos', 'naranjas']
```

Las listas son mutables

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Como agregamos los elementos en una lista?



añadir elementos en una lista

```
frutas = ['manzanas', 'peras', 'platanos', 'frutillas']  
print(frutas)  
frutas.append('naranjas')  
print(frutas)
```

```
['manzanas', 'peras', 'platanos', 'frutillas']  
['manzanas', 'peras', 'platanos', 'frutillas', 'naranjas']
```



añadir elementos en una posición específica a una lista

```
frutas = ['manzanas', 'peras', 'platanos', 'frutillas']  
print(frutas)  
frutas.insert(2, 'naranjas')  
print(frutas)
```

```
['manzanas', 'peras', 'platanos', 'frutillas']  
['manzanas', 'peras', 'naranjas', 'platanos', 'frutillas']
```

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Como eliminamos los elementos en una lista?



```
# eliminar elementos en una posición específica a una lista
frutas = ['manzanas', 'naranjas', 'peras', 'platanos', 'frutillas']
print(frutas)
del frutas[1]
print(frutas)
```

```
['manzanas', 'naranjas', 'peras', 'platanos', 'frutillas']
['manzanas', 'peras', 'platanos', 'frutillas']
```

```
# eliminar elementos con función pop
frutas = ['manzanas', 'naranjas', 'peras', 'platanos', 'frutillas']
print(frutas)
fruta_eliminada= frutas.pop()
print(frutas)
frutas.remove('peras')
print(frutas)
```

```
['manzanas', 'naranjas', 'peras', 'platanos', 'frutillas']
['manzanas', 'naranjas', 'peras', 'platanos']
['manzanas', 'naranjas', 'platanos']
```

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Como ordenamos los elementos en una lista?



- Generalmente, las listas se crean de manera **dinámica**, por lo que no siempre puedes controlar el orden en el que tus usuarios proporcionan sus datos.

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Como ordenamos los elementos en una lista?



```
▶ frutas = ['manzanas', 'peras', 'platanos', 'frutillas']  
print(frutas)  
frutas.sort()  
print(frutas)
```

```
['manzanas', 'peras', 'platanos', 'frutillas']  
['frutillas', 'manzanas', 'peras', 'platanos']
```

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Recorriendo los elementos de una lista?



```
1 frutas = ['manzanas', 'sandía', 'peras', 'platanos', 'frutillas']  
2 print(frutas)  
3 for i in frutas:  
4     print(i)
```

```
['manzanas', 'sandía', 'peras', 'platanos', 'frutillas']  
manzanas  
sandía  
peras  
platanos  
frutillas
```

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

Errores comunes con listas.



- Un error de índice (IndexError) significa que Python no puede averiguar el índice solicitado.
- Un error de tipo ValueError puede producirse al tratar de remover un valor que no se encuentra en la lista.

Mi computador es el mejor para repetir instrucciones.

¿Cómo manejamos más de una variable?, listas en Python.

```
[20] #errores comunes de indice
frutas = ['manzanas', 'peras', 'platanos', 'frutillas']
print(frutas)
print(frutas[-5])
```

```
['manzanas', 'peras', 'platanos', 'frutillas']
```

```
-----
IndexError                                Traceback (most recent call last)
<ipython-input-20-b86b898bb2dc> in <module>()
      1 frutas = ['manzanas', 'peras', 'platanos', 'frutillas']
      2 print(frutas)
----> 3 print(frutas[-5])
```

IndexError: list index out of range

SEARCH STACK OVERFLOW

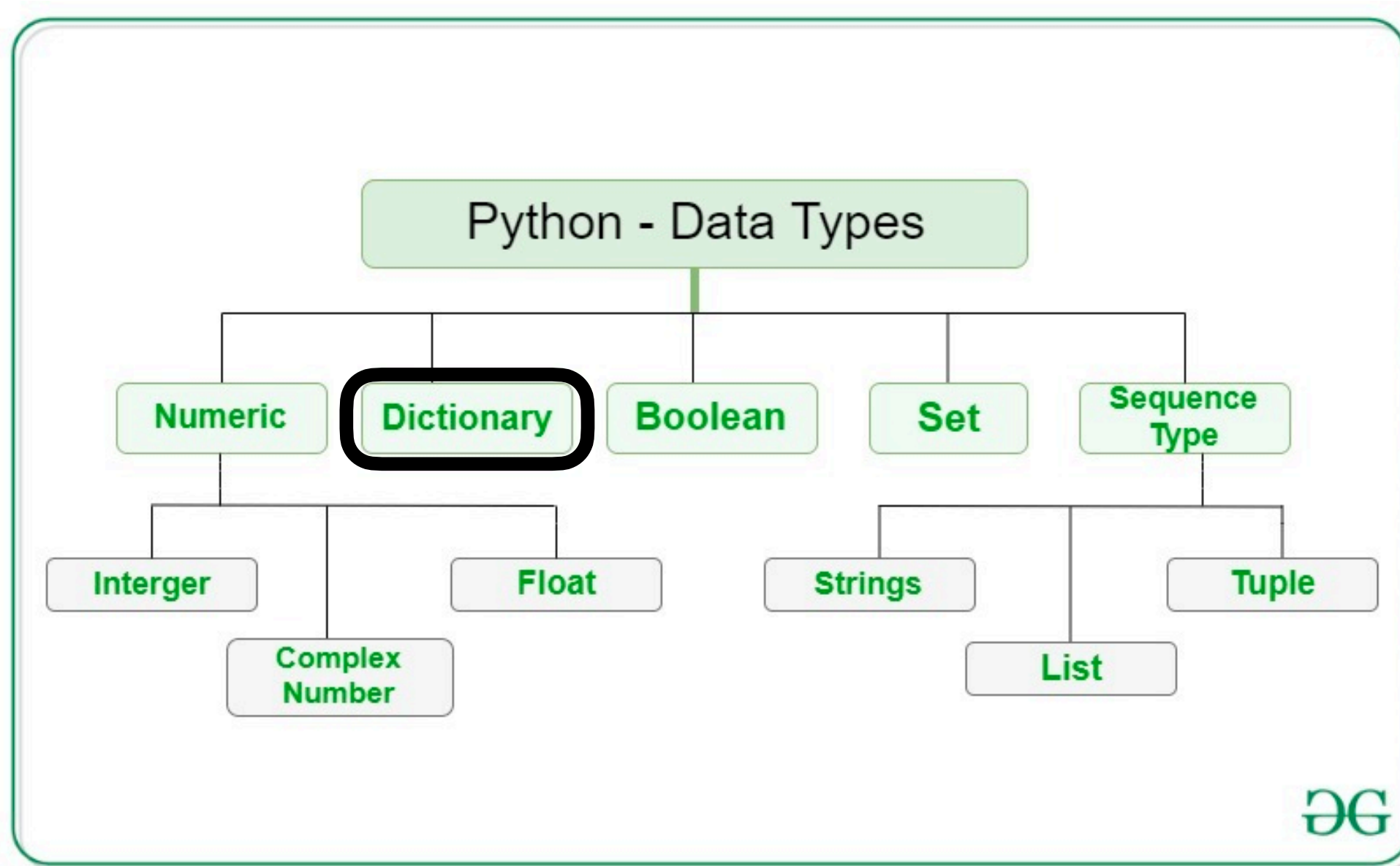
```
▶ #errores comunes de valor
frutas = ['manzanas', 'peras', 'platanos', 'frutillas']
print(frutas)
del frutas[5]
print(frutas)
```

```
☞ ['manzanas', 'peras', 'platanos', 'frutillas']
```

```
-----
IndexError                                Traceback (most recent call last)
<ipython-input-21-f0feb5a39f37> in <module>()
      2 frutas = ['manzanas', 'peras', 'platanos', 'frutillas']
```

Mi computador sabe cómo usar un diccionario.

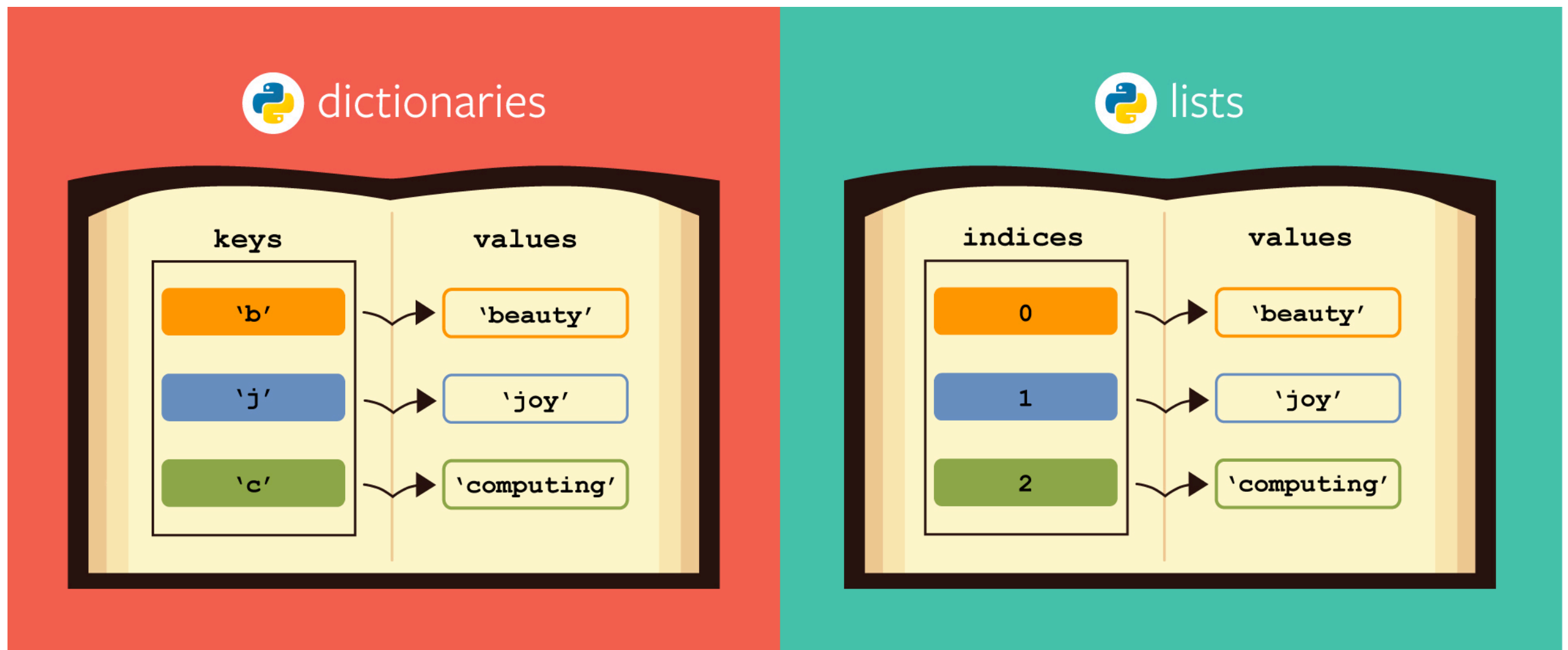
Presentación de diccionarios “hash” en python.



Mi computador sabe cómo usar un diccionario.

Presentación de diccionarios “hash” en python.

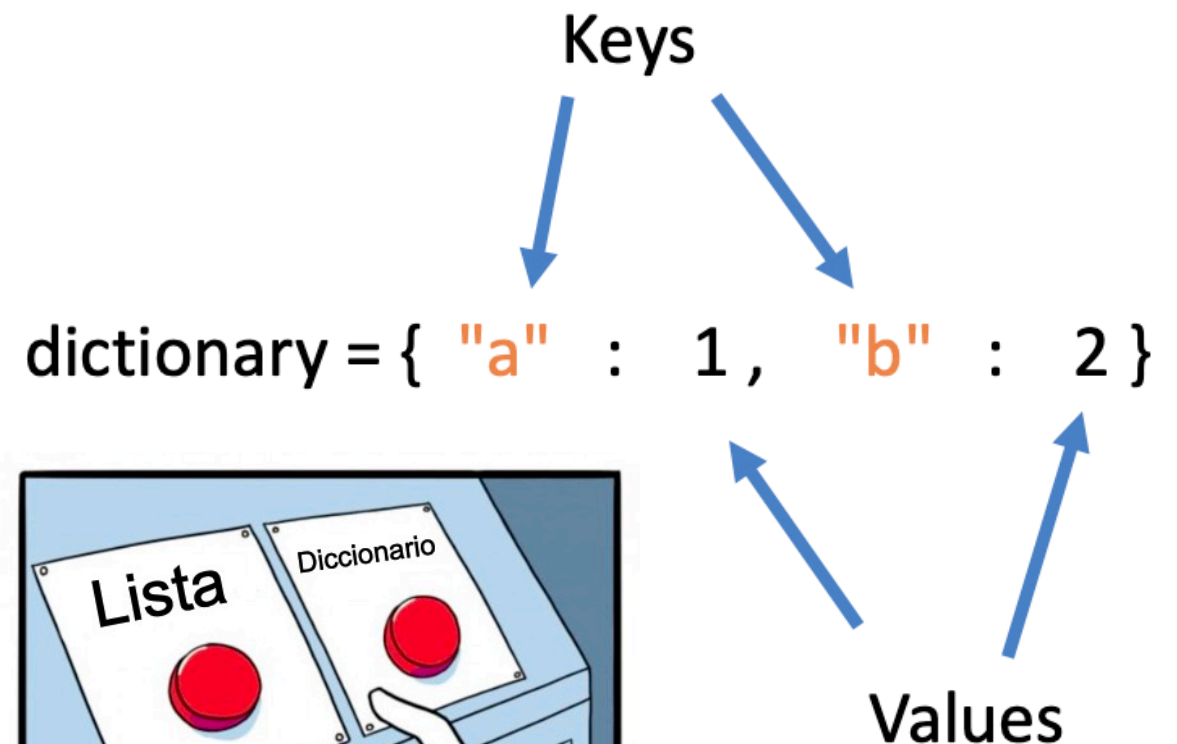
Qué es un diccionario?



Mi computador sabe cómo usar un diccionario.

Presentación de diccionarios “hash” en python.

- En Python, un diccionario es un tipo especial de lista.
- Clave: el nombre
- Valor: La cantidad o tipo de algo.



Mi computador sabe cómo usar un diccionario.

Presentación de diccionarios “hash” en python.

▼ Diccionarios vs listas

```
[27] 1 L=list()  
      2 n=10000000  
      3 #creamos una lista con 10M numeros  
      4 for i in range(n):  
      5     L.append(i)  
      6 # creamos un diccionario con 10M numeros  
      7 D=dict()  
      8 for i in range(n):  
      9     D[i]=str(i)
```

```
✓ [43] 1 import time  
7s    2 start = time.time()  
      3 f=0  
      4 for i in range(100):  
      5     if random.randint(0,n) in L:  
      6         f=f+1  
      7 end = time.time()  
      8 print("Lista",end - start)
```

Lista 7.328480243682861

**Diccionario es ~14000
veces más rapido que
la lista**

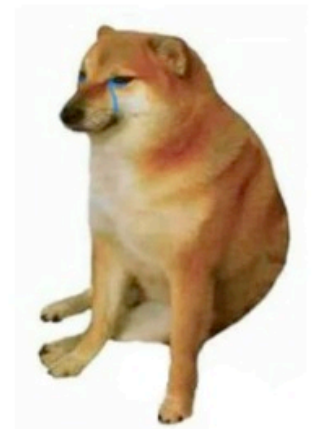
```
✓ [43] 1 import time  
0s    2 start = time.time()  
      3 f=0  
      4 for i in range(100):  
      5     if random.randint(0,n) in D.keys():  
      6         f=f+1  
      7 end = time.time()  
      8 print("Diccionario",end - start)
```

☞ Diccionario 0.0005049705505371094

Lista



Guardar



Buscar



Mi computador sabe cómo usar un diccionario.

Operaciones con diccionarios (agregar, buscar, borrar) en Python.

- clear
- copy
- fromkeys
- get
- items
- keys
- pop
- popitem
- setdefault
- update
- values

```
1 #crear diccionario
2 D=dict()
3 NP={'carlos':1234,'roberto':6655,'camila':1414,'andrea':1212}
4 #acceder por clave
5 print(NP['andrea'])
6 #borra el diccionario
7 NP.clear()
8 NP={'carlos':1234,'roberto':6655,'camila':1414,'andrea':1212}
9 #copiar un diccionario
10 D=NP.copy()
11 print(D)
12 #obtener el valor por clave
13 print(NP.get("roberto"))
14 #items
15 print(NP.items())
16 #claves
17 print(NP.keys())
18 #valores
19 print(NP.values())
20 #eliminar valores por clave
21 a = NP.pop("camila")
22 print(NP, a)
```

```
1212
{'carlos': 1234, 'roberto': 6655, 'camila': 1414, 'andrea': 1212}
6655
dict_items([('carlos', 1234), ('roberto', 6655), ('camila', 1414), ('andrea', 1212)])
dict_keys(['carlos', 'roberto', 'camila', 'andrea'])
dict_values([1234, 6655, 1414, 1212])
{'carlos': 1234, 'roberto': 6655, 'andrea': 1212} 1414
```

Mi computador sabe cómo usar un diccionario.

Operaciones con diccionarios (agregar, buscar, borrar) en Python.

```
# crear bd usuarios
NP={ 'carlos':1234, 'roberto':6655, 'camila':1414, 'andrea':1212}
# obtener usuario y password
nombre=input("ingrese nombre:")
password=int(input("ingrese pass"))
if nombre in NP.keys():
    if NP[nombre] == password:
        print("Welcome", nombre)
    else:
        print("Password incorrecto")
else:
    print("Usuario no registrado")
```

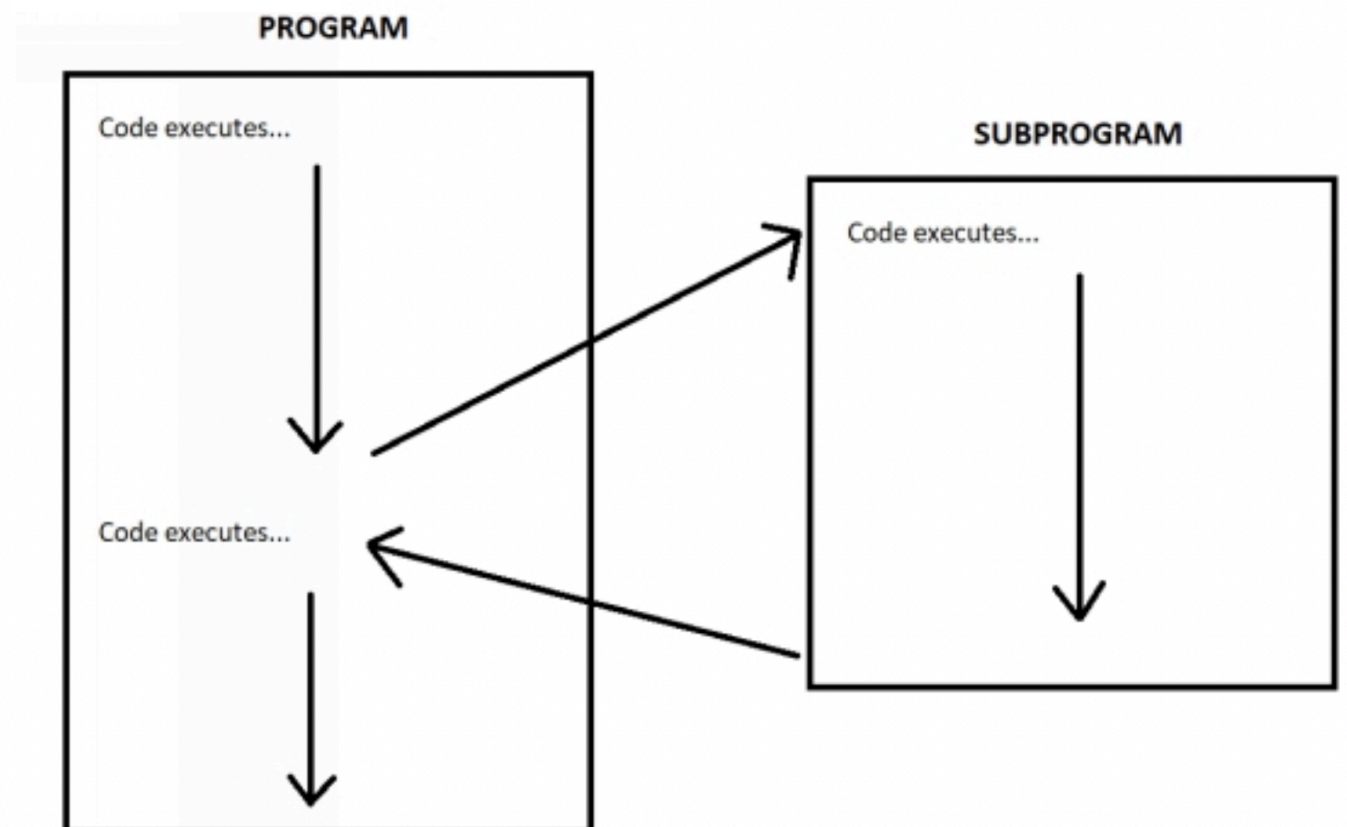
Que hace este código?

¿Puedo agregar funciones a mi computador?

No es necesario programar la rueda una y otra vez (concepto función)

- Una función es el trabajo de alguien.
 - Ejemplo : Bomberos combatir incendios.
- En programación una función es un bloque de código que hace una cosa particular.
- Lo interesante de las funciones es que se pueden reutilizar.
- Para hacer una función en Python usamos el comando **def**.
 - Nombre parametros return
- Funciones también se llaman subprogramas

```
def restar(a,b):  
    return a - b  
#llamando la nueva funcion  
print(restar(5,2))  
print(restar(9,5))
```



¿Puedo agregar funciones a mi computador?

No es necesario programar la rueda una y otra vez (concepto función)

```
1 def xxx(n):  
2     i=0  
3     t=0  
4     while i < n:  
5         t=t+i  
6         i=i+1  
7  
8     return t  
9  
10 print(xxx(10))  
11 print(xxx(6))
```

➞ 45
15

```
1 def zzzz(lista):  
2     total=0  
3     for i in lista:  
4         total=total+i  
5     return total/len(lista)  
6  
7 L=[1,2,5,6,9,10]  
8  
9 print(zzzz(L))
```

➞ 5.5

```
1 def yyyy(lista):  
2     m = -1  
3     for i in lista:  
4         if(i > m):  
5             m=i  
6     return m  
7  
8 print(yyyy(L))
```

➞ 10

Que hacen las funciones?

Una serpiente que programa?

GitHub

The screenshot displays the GitHub interface for the repository `adigenova/tpp`. The repository is public and contains the following files and folders:

File/Folder	Commit Message	Time Ago
clases	adding class 1	4 hours ago
code	Created using Colaboratory	5 hours ago
programa	adding programa	8 hours ago
README.md	Update README.md	8 hours ago

The `README.md` file contains the following text:

Todas y Todos Podemos Programar (TTP)

Curso Introductorio de Python para alumnos de Tercero y cuarto Medio.

Programa del curso

Programa que detalla el contenido de cinco clases teoricas y cinco talleres prácticos.

Bitacora de clases

Archivo guía con el contenido y código publicados en el repositorio github del taller.

The right sidebar shows repository statistics:

- Readme
- 0 stars
- 1 watching
- 0 forks
- No releases published
- No packages published
- 100.0% Jupyter Notebook

<https://github.com/adigenova/tpp>